Bit-slice Architecture for DNN Acceleration with Slice-level Sparsity Enhancement and Exploitation

Insu Choi†, Young-Seo Yoon†, and Joon-Sung Yang†‡
†Department of Electrical and Electronic Engineering, ‡Department of Systems Semiconductor Engineering
Yonsei University, Seoul, South Korea
{insuofficial, ys.yoon, js.yang}@yonsei.ac.kr

2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00067

***Abstract*—Deep Neural Networks (DNNs) demand significant computational resources, prompting the emergence of bit-slice architectures designed to efficiently accelerate DNNs by leveraging high bit-precision reconfigurability and fine-grained sparsity through slice-level computation. However, fully utilizing slice-level sparsity remains challenging, leading conventional bit-slice architectures to exploit either input or weight sparsity at a coarse-grained level. In this paper, we introduce a *Bit-slice Architecture for DNN Acceleration* (BADA) that simultaneously leverages both input and weight sparsity at coarse- and fine-grained levels. BADA features a novel architecture that skips computations for bit-slice chunks containing zero values and also skips any set of operands where either the input or weight bit-slice is zero. The design comprises a front-end unit responsible for generating bit-slices and selectively gathering only the non-zero slices, and a back-end unit equipped with a signed multiply-and-accumulate (MAC) unit to process these collected non-zero bit-slices. Additionally, we present two algorithmic optimizations to further enhance the efficiency and performance of BADA. First, we propose a novel bit-slice representation that supports 8-bit data without incurring additional hardware overhead, whereas conventional bit-slice representations are limited to 6-bit or 7-bit data under similar constraints. Second, we introduce a method to narrow the weight distribution during the training process, thereby increasing the proportion of zero-valued higher-order bit-slices and further enhancing slice-level weight sparsity. Experimental results demonstrate that BADA achieves a $2.67\times$ increase in throughput, a $1.52\times$ improvement in area efficiency, and a $2.15\times$ enhancement in energy efficiency compared to LUTein, the state-of-the-art bit-slice architecture.**

I. Introduction

Deep Neural Networks (DNNs), encompassing advanced models such as Convolutional Neural Networks (CNNs) [27], [63], [68], [72] and Transformer-based models [6], [12], [13], [48], [75], [76], have surpassed human performance in numerous applications, including computer vision and natural language processing (NLP) tasks. However, DNNs demand substantial computational resources, particularly for multiply-and-accumulate (MAC) operations. Consequently, a variety of DNN accelerators [7], [37], [44], [53], [55], [79] and optimization techniques [15], [16], [18], [29], [35], [43], [51] have been extensively explored. Among these optimization techniques, quantization stands out as a method that reduces the bit precision of DNN models from higher to lower levels, thereby decreasing computational and memory requirements.

Alongside these quantized DNN models, bit-slice architectures [23], [33], [34], [66] have been introduced to support reconfigurable bit precision through slice-level computation. These architectures decompose integer data into multiple bit-slices, which are then processed using numerous low-precision integer MAC units in either a temporal or spatial manner. Decomposing integer data into bit-slices not only supports various reduced bit precisions but also creates new opportunities to exploit slice-level sparsity for additional performance gains.

However, conventional bit-slice architectures face limitations in fully leveraging slice-level sparsity. For instance, [66] does not exploit sparsity, whereas [23] leverages slice-level input sparsity exclusively. State-of-the-art bit-slice architectures [33], [34] utilize either slice-level input or weight sparsity, but do so in a coarse-grained manner. Moreover, these architectures require multipliers with bit-widths exceeding the actual data precision: [66] and [23] employ 5-bit multipliers for 4-bit bit-slices, while [33] and [34] use 4-bit multipliers for 3-bit bit-slices by introducing the Signed Bit-slice Representation (SBR).

In this paper, we introduce a novel *Bit-slice Architecture for DNN Acceleration* (BADA) that addresses the limitations of conventional bit-slice architectures. BADA is designed to fully exploit slice-level sparsity by simultaneously leveraging both input and weight sparsity at coarse- and fine-grained levels. Specifically, it reduces computational overhead by skipping operations on chunks of zero bit-slices or when either the input or weight bit-slice is zero. To achieve this, we present an innovative architecture comprising two main components: (i) a front-end unit that generates 4-bit bit-slices from 8-bit data and selectively aggregates only the non-zero bit-slices, and (ii) a back-end unit equipped with a signed MAC unit to process these collected non-zero bit-slices. Additionally, we propose a novel bit-slice representation that splits an 8-bit signed integer into two 4-bit signed integers by incorporating either the sign bit or the carry bit of the lower-order bit-slice into the higher-order bit-slice. The proposed bit-slice representation enables BADA to use 4-bit multipliers when operating on two 8-bit integers, whereas conventional bit-slice architectures require 5-bit or 4-bit multipliers for 8-bit or 7-bit data, respectively. Furthermore, we introduce scale regularization, a regularization method applied to the loss function during model training. Training the model with scale regularization narrows the weight distribution and effectively increases the proportion of near-zero weight values with zero-valued higher-order bit-slices, thereby enhancing slice-level weight sparsity.

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00067

We evaluate BADA, along with these algorithmic optimizations, on both vision and NLP tasks. For image classification, we employ models such as VGG-16 [68], ResNet-50 [27], Inception-v3 [72], DenseNet-121 [32], MobileNetV2 [63], and DeiT [75], all trained and tested on the ImageNet dataset [40]. For NLP tasks, we evaluate RoBERTa [48] on the General Language Understanding Evaluation (GLUE) benchmark [78], and Large Language Models (LLMs) such as Llama-3.2 1B [14] and Gemma-2 2B [74] on the HellaSwag benchmark [88]. Experimental results show that BADA achieves substantial improvements over LUTein [34], including enhancements of $2.67\times$ in throughput, $1.52\times$ in area efficiency, and $2.15\times$ in energy efficiency.

II. Background

A. Quantization

Quantization is a technique employed to reduce the bit-width of numerical values. In DNNs, quantization transforms inputs and weights from their original 32-bit floating-point representations to lower-bit precisions, typically 8-bit integers. This reduction in bit-width significantly decreases the computational overhead of matrix multiplication, resulting in a reduction by a factor of 16 in computational complexity and a reduction by a factor of 4 in memory usage. However, decreasing the bit-width can introduce quantization noise, which may adversely affect the accuracy of DNNs.

To mitigate accuracy loss, various quantization methods have been studied [15], [35], [43], [49]. These methods can be classified into two categories: Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT). PTQ is a lightweight quantization method that does not require a training process and necessitates only a small amount of data, or no data at all. In contrast, QAT relies on fine-tuning DNNs using training data while applying simulated quantization operations during the training process [35]. Although QAT requires significant engineering effort during model training, it enables the production of lower-bit precision models while maintaining accuracy in most cases, even for models that cannot be effectively quantized using PTQ.

In this paper, we employ LSQ [15], a QAT method, to quantize DNN models. LSQ is well-suited for our novel algorithmic method because one of its key features is the training of scale factors, which are crucial for determining the step size of quantization. By integrating our proposed method with LSQ, we encourage the scale factors to be larger and narrow the distribution of quantized weights, thereby enhancing sparsity, as detailed in Section III-B.

B. Bit-slice Architectures

The widespread adoption of quantization as an optimization method has driven the development of various bit-slice architectures. The foundational bit-slice architecture [66] supports reconfigurable bit precision by employing low bit-precision multipliers and shifting logic, thereby enabling mixed-precision multiplications. This flexibility enhances performance across a wide spectrum of DNN models with diverse bit-precision requirements. In [23], performance is further improved by exploiting slice-level input sparsity, which omits computations for bit-slices associated with zero inputs and extends this omission to higher-order bit-slices corresponding to near-zero positive inputs.

State-of-the-art bit-slice architectures [33], [34] exploit either slice-level input or weight sparsity by monitoring data sparsity at runtime and dynamically deciding which sparsity to utilize. Moreover, [33] and [34] adopt SBR, enabling the use of 4-bit multipliers instead of conventional 5-bit multipliers. However, they utilize sparsity in a coarse-grained manner through a compression scheme with a granularity of four bit-slices, allowing computations to be skipped only when all four bit-slices in a window are zero. In addition, although 4-bit multipliers are employed with SBR, the actual processing is performed with 3-bit bit-slices rather than 4-bit bit-slices.

The proposed bit-slice architecture, BADA, emphasizes the fine-grained utilization of both input and weight sparsity. By leveraging these two forms of sparsity, BADA can skip computations whenever either the input or weight bit-slice is zero, yielding significant performance gains. In addition to fine-grained sparsity, BADA also exploits coarse-grained sparsity by omitting computations for groups of zero bit-slices. Furthermore, its novel bit-slice representation allows 4-bit multipliers to process 4-bit bit-slices, thereby expanding the representational range of the original data (e.g., 8-bit or 16-bit) compared to SBR (e.g., 7-bit or 13-bit), without incurring additional hardware overhead. Finally, our algorithmic optimization that increases slice-level weight sparsity further enhances the performance of BADA.

III. Algorithmic Optimization

A. Bit-slice Representation

In the conventional bit-slice architectures [23], [66], the bit-slice representation decomposes two's complement integer data into both signed and unsigned bit-slices. The highest-order bit-slice is treated as signed, while the remaining bit-slices are treated as unsigned. Consequently, these architectures require a sign-extension unit and higher bit-precision multipliers (e.g., 5-bit multipliers to compute 4-bit bit-slices). In contrast, SBR [33] decomposes m-bit data ($m = 4, 7, 10, 13, \ldots$) into a highest-order bit-slice of 4 bits and lower-order bit-slices of 3 bits each, thereby enabling the use of 4-bit multipliers for both 4-bit and 3-bit bit-slices without requiring a sign-extension unit. Additionally, SBR converts the higher-order bit-slices of negative near-zero values into zeros, enhancing slice-level sparsity. However, SBR still necessitates redundant bit-precision multipliers, as 4-bit multipliers are used to compute 3-bit bit-slices. To address the limitations of conventional bit-slice representations, we propose a novel bit-slice representation that decomposes n-bit data ($n = 4, 8, 12, 16, \ldots$) into 4-bit bit-slices and transforms the higher-order bit-slices of negative near-zero values into zeros, all while maintaining the same hardware requirements as conventional bit-slice architectures.

The proposed bit-slice representation is achieved by adding the sign bit or carry bit of a lower-order bit-slice to the next higher-order bit-slice. Fig. 1 illustrates the process of converting a 16-bit integer into four 4-bit bit-slices using the proposed representation, demonstrated with an example value of 18430. The 16-bit data is first decomposed into four 4-bit intermediate representations, IR_3, IR_2, IR_1, and IR_0. The lowest-order intermediate representation, IR_0, is retained in the lowest-order bit-slice. The subsequent bit-slice is generated by adding the sign bit from the lowest-order bit-slice to IR_1, which produces a carry and a sum of 0000_2. Notably, the carry bit and the sign bit cannot both be 1_2 simultaneously. Consequently, if either the carry or the sign bit is 1_2, a value of 1_2 is added to the next intermediate representation. In the example shown in Fig. 1, the carry bit from the second bit-slice is added to IR_2 to produce the third bit-slice, resulting in 1000_2. This iterative process continues until the final bit-slices are obtained, which are $0101_2(5)$, $1000_2(-8)$, $0000_2(0)$, and $1110_2(-2)$. In contrast, a conventional bit-slice representation would involve sign-extending the intermediate representations, resulting in $00100_2(4)$, $00111_2(7)$, $01111_2(15)$, and $01110_2(14)$.

To validate the correctness of the proposed bit-slice representation, we demonstrate the restoration of the original value from the final bit-slices shown in Fig. 1 as follows:

$$(5 \times 2^{12}) + (-8 \times 2^{8}) + (0 \times 2^{4}) + (-2 \times 2^{0}) = 18430 \quad (1)$$

This result is equivalent to the restoration achieved with sign-extended bit-slices using the conventional representation:

$$(4 \times 2^{12}) + (7 \times 2^{8}) + (15 \times 2^{4}) + (14 \times 2^{0}) = 18430 \quad (2)$$

When negative numbers are provided, the proposed bit-slice representation can be applied using the same methodology as for positive numbers. For example, consider the binary value $1111\_1111\_1111\_1000_2(-8)$. The lowest-order bit-slice is first identified as $1000_2(-8)$. Next, the sign bit of this lowest-order bit-slice is added to the subsequent intermediate representation 1111_2, yielding $0000_2(0)$ along with a carry bit. This carry bit is then added to another subsequent 1111_2, generating an additional carry. Repeating this process ultimately decomposes the binary value $1111\_1111\_1111\_1000_2(-8)$ into three bit-slices with $0000_2(0)$ and one bit-slice with $1000_2(-8)$. Notably, the higher-order bit-slices become zero, thereby enhancing slice-level sparsity.

To analyze the proposed bit-slice representation, consider an n-bit two's complement integer I defined by

$$I = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i \quad (3)$$

where $b_i \in \{0, 1\}$ denotes the i-th bit of I. We expand Equation (3) and group the terms in blocks of four bits to form 4-bit bit-slices. Additionally, in each block (except for the highest-order block), the highest term is converted to its negative form by subtracting and re-adding twice its value:

$$\begin{aligned} I = &-b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + b_{n-3}2^{n-3} + b_{n-4}2^{n-4} \\ &+ 2(b_{n-5}2^{n-5}) - 2(b_{n-5}2^{n-5}) \\ &+ b_{n-5}2^{n-5} + b_{n-6}2^{n-6} + b_{n-7}2^{n-7} + b_{n-8}2^{n-8} \\ &+ \cdots \\ &+ 2(b_3 2^3) - 2(b_3 2^3) \\ &+ b_3 2^3 + b_2 2^2 + b_1 2^1 + b_0 2^0 \end{aligned} \quad (4)$$

We define a 4-bit signed integer S_m to include the bits from positions m through $m+3$ of I, as follows:

$$S_m = -b_{m+3}2^3 + \sum_{i=0}^{2} b_{m+i}2^i \quad (5)$$

Hence, each group in Equation (4) corresponds to one 4-bit signed integer S_m, which can be summarized by

$$I = (S_{n-4} + b_{n-5})2^{n-4} + (S_{n-8} + b_{n-9})2^{n-8} + \cdots + S_0 \quad (6)$$

indicating that I can be represented by multiple signed bit-slices, each comprising a signed integer S_m added to the sign bit of the lower-order integer b_{m-1}. Furthermore, if S_m is $0111_2(7)$ and b_{m-1} is 1_2, the result is $1000_2(-8)$ with a carry of 2^{m+4}, which is added to S_{m+4}. These arithmetic operations can be performed by the algorithm illustrated in Fig. 1.

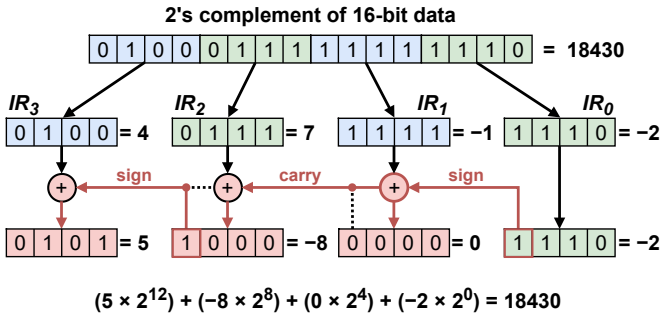


Fig. 1. Example of the proposed bit-slice representation.

While the proposed bit-slice representation effectively decomposes I into signed bit-slices, its representational range changes due to overflow when it includes higher-order bit-slices of 0111_2 and a lowest-order bit-slice of 1000_2. For example, the 8-bit integer $0111\_1000_2(120)$ is converted into two bit-slices of $1000_2(-8)$, which cannot be restored to the original value of 120 because $(-8 \times 2^4) + (-8 \times 2^0) = -136$. Consequently, we introduce the *Range-Shifted Integer* (RS-INT), whose representational range is shifted by Δ relative to the original integer (INT). Formally, Δ is defined as follows:

$$\Delta = \sum_{i=0}^{n/4-2} 2^{4i+3} \quad (7)$$

The range of RS-INT becomes $\left[-2^{n-1} - \Delta, 2^{n-1} - 1 - \Delta\right]$. By employing RS-INT, numerical overflow is effectively prevented when converting data into bit-slices using the proposed bit-slice representation.

B. Narrow Weight Distribution

The proposed bit-slice representation converts the higher-order bit-slices of near-zero data to zero, enabling slice-level sparsity that can be exploited by hardware for improved performance. This feature is particularly advantageous for DNN weights, which typically follow a Gaussian distribution and thus exhibit numerous near-zero values. By forming a distribution narrower than the Gaussian baseline, the number of near-zero weight values can be further increased, thereby enhancing slice-level weight sparsity. Consequently, we propose a method that narrows the weight distribution, forming a Narrow Weight Distribution (NWD), which increases slice-level sparsity and can be effectively exploited by the proposed architecture to further enhance overall performance.

NWD is achieved by controlling the scale factor s, a critical parameter in the quantization process that specifies the step size of the quantizer, defined as

$$x_{int} = clip\left(\left\lfloor \frac{x}{s} \right\rceil, \alpha, \beta\right) \tag{8}$$

where $clip(z, a, b)$ clamps z to the range $[a, b]$, and $\lfloor z \rceil$ rounds z to the nearest integer. The values α and β represent the minimum and maximum of the target n-bit integer range, respectively. In this paper, we employ the RS-INT format, setting α to $-2^{n-1} - \Delta$ and β to $2^{n-1} - 1 - \Delta$. To maintain the accuracy of the quantized DNN model, an optimal scale factor s must be determined to minimize quantization error. However, to narrow the distribution of quantized weights and form an NWD, s must be as large as possible, mapping the original values to smaller integer values than would be the case under the optimal scale factor. Although a larger scale factor s introduces greater quantization noise, empirical results indicate that model accuracy remains resilient up to a certain noise threshold.

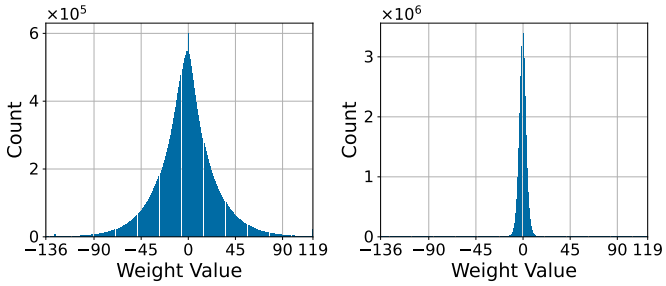
To encourage larger scale factors, we introduce *scale regularization*, a regularization method applied to the loss function $\mathcal{L}$ during LSQ [15], a Quantization-Aware Training (QAT) method that learns scale factors during model training, as follows:

$$\mathcal{L} = \mathcal{L}_T + \lambda \mathcal{L}_S \tag{9}$$

where $\mathcal{L}_T$ is the target loss function, $\mathcal{L}_S$ is the proposed scale regularization term, and λ is a coefficient controlling the regularization effect. Considering a quantization setting that includes per-channel (per-column) quantization for weights and per-tensor quantization for inputs [83], the scale regularization term $\mathcal{L}_S$ is defined as

$$\mathcal{L}_S = \sum_{l=1}^{L} \sum_{c=1}^{C} \frac{1}{\sqrt{s_c^l}} \tag{10}$$

where L is the number of layers, C is the number of channels (columns) in each layer's weights, and s_c^l denotes the scale factor for the c-th channel (column) in the l-th layer. The scale regularization penalizes the inverse of the scale factor in Equation (10), driving scale factors to larger values because the penalty is minimized when scale factors increase. Furthermore, applying the square root prevents the penalty term from becoming excessively large. Consequently, by employing scale regularization in conjunction with LSQ, the scale factors across all layers are trained to be as large as possible, thereby forming an NWD while preserving model accuracy comparable to that attained with optimally trained scale factors.



(a) Baseline (b) NWD

Fig. 2. Distribution of quantized weights in ResNet50. (a) Baseline weights and (b) weights trained with scale regularization, resulting in a Narrow Weight Distribution (NWD).

Fig. 2 illustrates the distribution of quantized weights in ResNet50 [27] when trained without (Fig. 2a) and with (Fig. 2b) scale regularization, using a λ of 1e-6. The minimum and maximum weight values (-136 and 119, respectively) reflect the use of 8-bit RS-INT. Introducing the regularization raises the average scale factor from 2.32 to 200.60, thereby yielding an NWD, as shown in Fig. 2b. Consequently, the proportion of weights within the range $[-8, 7]$ increases, which expands the number of weights featuring zero-valued higher-order bit-slices. This heightened slice-level weight sparsity, rising from 19.1% to 49.9%, incurring only a negligible accuracy drop of 0.08% in our ResNet50 experiments (detailed in Section V-B and V-C).

IV. BADA Architecture

A. Overall Architecture

The overall architecture of BADA is depicted in Fig. 3. It consists of a BADA core, a Multipurpose Processing Unit (MPU), and on-chip shared buffers. The BADA core contains N Processing Element Arrays (PE Arrays), each composed of four Processing Elements (PEs). The MPU includes high-precision fixed-point multipliers and adders for quantization, activation functions (including softmax), batch/layer normalization, and residual operations. The on-chip shared Input Buffer (IBUF), Weight Buffer (WBUF), and Output Buffer (OBUF) have capacities of 128 KB, 128 KB, and 64 KB, respectively. These buffers are shared among the PEs and the MPU and interface with external memory.

BADA processes General Matrix Multiplication (GEMM) operations using a tiling strategy. First, BADA fetches input and weight matrices from external memory, storing them in the shared IBUF and WBUF, respectively. Each PE Array then loads the tiled matrices to produce an 8×8 output matrix,

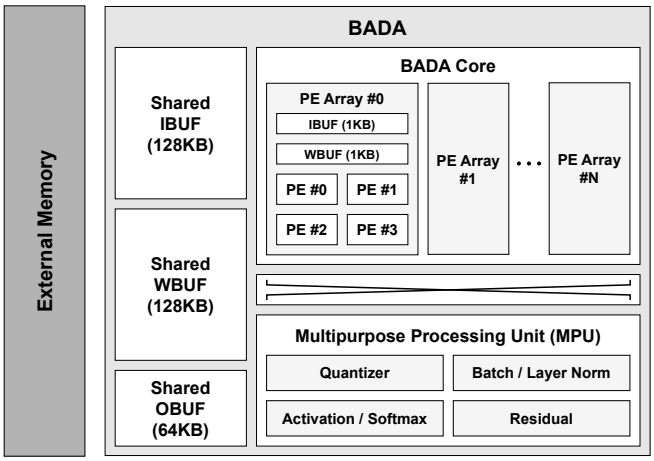


Fig. 3. Overall architecture of BADA.

while each PE within the PE Array generates a 4×4 output matrix. To optimize data I/O operations, the write-back of the output matrix to the shared OBUF is performed with a granularity of 4×4 output matrices, requiring four cycles to write back the 8×8 output matrix processed in the PE Array. Since each 4×4 output matrix is processed in a PE that uses 16 individually operated MAC Units in the back-end unit, as shown in Fig. 4, a synchronization operation is performed once each 4×4 output matrix is generated in the PE. This dataflow approach maximizes the reuse of matrix vectors in both the PE Array and the PE, thereby reducing memory overhead. Notably, the output matrices consist of high-precision integer values, which are then processed for activation functions, softmax, batch/layer normalization, and residual operations within the MPU, according to the requirements of the DNN model. Finally, the processed output matrices are quantized to 8-bit and written back to the shared IBUF, preparing them for the next layer of the DNN model.

B. Processing Element

The Processing Element (PE) incorporates an efficient hardware mechanism to leverage the proposed bit-slice representation and exploit both coarse-grained and fine-grained slice-level input and weight sparsity. The PE microarchitecture consists of two distinct components: a front-end unit and a back-end unit, as illustrated in Fig. 4. The front-end unit includes Bit-slice Units and Non-zero Bit-slice Collectors (NBCs), which generate bit-slices and collect only non-zero bit-slices to fully exploit both structured and unstructured sparsity. In contrast, the back-end unit comprises MAC Units responsible for computing the collected non-zero bit-slices.

The dataflow begins by loading 16 sets of 8-bit input and weight data from the IBUF and WBUF of the PE Array. Subsequently, the 8-bit data are transformed into 4-bit bit-slices within the corresponding Bit-slice Unit. The Input Higher-Order Bit-slices (IHOBs) and Input Lower-Order Bit-slices (ILOBs), which are outputs of the Bit-slice Unit, are stored in the IHOB BUF and ILOB BUF, respectively. Similarly, the

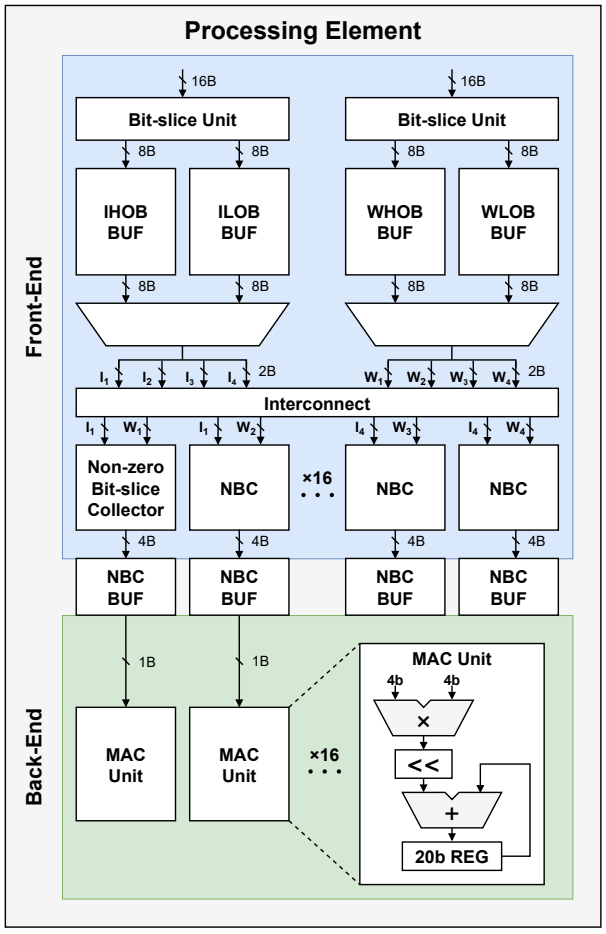


Fig. 4. Microarchitecture of the Processing Element.

Weight Higher-Order Bit-slices (WHOBs) and Weight Lower-Order Bit-slices (WLOBs) are stored in the WHOB BUF and WLOB BUF. To compute results corresponding to the original 8-bit data, each IHOB or ILOB must be paired and computed with both WHOBs and WLOBs. Therefore, the controller selects outputs from the IHOB or ILOB BUF and the WHOB or WLOB BUF through a multiplexer (MUX) to obtain four combinations of phases per clock cycle: IHOB/WHOB, IHOB/WLOB, ILOB/WHOB, and ILOB/WLOB.

All BUFs are equipped with flag bits that indicate whether all 16 sets of 4-bit bit-slices are zero-valued, operating at an 8-byte granularity. The controller examines the flag bit at the beginning of each phase and skips any phases for which the corresponding flag bit is active. For instance, if the flag bit of the IHOB BUF is set to 1, the controller skips the IHOB/WHOB and IHOB/WLOB phases and immediately advances to the next phase (ILOB/WHOB) by controlling the MUX. These flag bits enable coarse-grained sparsity utilization by omitting chunks of zero bit-slices, thereby reducing the latency of operand generation in the front-end unit. Consequently, as the ratio of operands generated in the front-end to those consumed in the back-end unit increases, the utilization of the back-end unit improves significantly.

The selected bit-slices from the BUFs are divided into four groups, with each group forming a distinct row or column vector of the input or weight matrix. A total of 16 combinations from the four input groups (I_1, I_2, I_3, and I_4) and four weight groups (W_1, W_2, W_3, and W_4) are fed into 16 Non-zero Bit-slice Collectors (NBCs). The separation of groups allows the PE to reuse vectors when generating a 4×4 output matrix, and the dot products are computed in a temporal manner in each NBC and MAC unit.

NBC examines both input and weight bit-slices, collecting only the non-zero bit-slices to feed into the back-end stage. Specifically, NBC has a 4-byte bit-width for both its input and output data buses since all bit-slices are preserved when dense data is provided. However, when sparse data is given, NBC directs the non-zero bit-slices to one side of the data bus and communicates the occupied range of the data bus to the controller. Subsequently, the data from NBCs are stored in NBC Buffers (NBC BUFs).

Finally, the back-end unit consumes non-zero bit-slices at a 1-byte granularity. It consists of a 4-bit multiplier and an accumulator with shifting logic, implementing MAC operations on the non-zero bit-slices collected in the front-end unit. Owing to the proposed bit-slice representation, 4-bit multipliers are employed. During the IHOB/WHOB phase, the multiplier output is left-shifted by 8 bits, and by 4 bits during the IHOB/WLOB and ILOB/WHOB phases.

C. Bit-slice Unit

The Bit-Slice Unit (BSU) in the PE is a critical component responsible for converting 8-bit data into bit-slices using the proposed bit-slice representation. Since each 8-bit datum is split into two 4-bit bit-slices, the BSU produces one pair of bit-slices for each input datum. The BSU microarchitecture is illustrated in Fig. 5. The LOB is formed by directly extracting the four Least Significant Bits (LSBs) of the input 8-bit datum. In contrast, the HOB is generated through a controlled adder that takes the four Most Significant Bits (MSBs) and the sign bit of the LOB as inputs. The sign bit of the LOB passes through an AND gate controlled by a signal (`ctrl`) that enables or disables its addition to the MSBs. By default, `ctrl` is set to 1, allowing the addition. However, when the input data is 4-bit rather than 8-bit, `ctrl` is set to 0, effectively bypassing the addition. This mechanism, controlled by the `ctrl` signal, supports reconfigurability between 4-bit and 8-bit operations. To accommodate the parallel processing of 16 8-bit data elements, the BSU is designed with a 16-byte data bus. Consequently, a total of 16 adders are included to enable the simultaneous conversion of 16 8-bit data elements into their corresponding bit-slice representations.

D. Non-zero Bit-slice Collector

The Non-zero Bit-slice Collector (NBC) plays a crucial role in efficiently omitting computational operations associated with zero bit-slices, thereby achieving fine-grained utilization of slice-level input and weight sparsity. The NBC examines pairs of Input Bit-slices (IBs) and Weight Bit-slices (WBs) to determine whether both IB and WB are non-zero. Non-zero pairs are then collected and placed on one side of the data bus, while the remaining bus portion is filled with zero values that the back-end unit does not process. Additionally, to ensure that only the collected non-zero pairs are processed in the back-end unit, the NBC provides the controller with information on the number of non-zero pairs, thereby omitting pairs that contain zero IB or WB from computation.

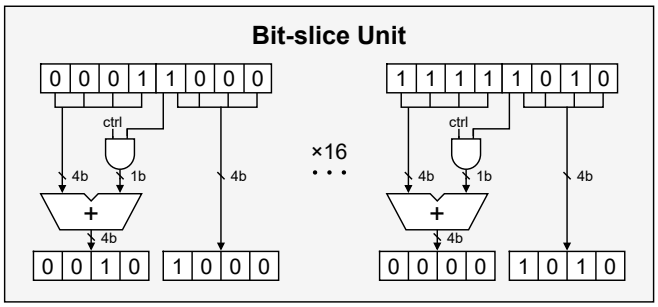


Fig. 5. Microarchitecture of the Bit-slice Unit (`ctrl` = 1).

Fig. 6 illustrates the microarchitecture of the NBC, which primarily comprises MUXs and MUX Input blocks responsible for providing input data and select signals to the MUXs. The MUX Input block contains Non-zero Detectors that verify whether either IB or WB is non-zero and generate a corresponding select signal. Initially, the input pairs IB_1/WB_1 through IB_4/WB_4 are fed into the MUX Input block. The MUX Input block concatenates each IB and WB to form a DATA line and produces the select signal (SEL) for the first row of MUXs.

Each DATA line is shared among the input 1 of the MUXs in a column-wise arrangement, with the number of MUXs increasing by one in each subsequent column. For instance, the DATA line produced by the IB_1/WB_1 pair is shared with one MUX, whereas the line from the IB_2/WB_2 pair is distributed to two MUXs. In contrast, the input 0 of the MUXs is sourced from the output of the previous MUX in a row-wise manner, except for the leftmost MUXs, whose input is a zero value.

The SEL signals connect to the MUXs through simple logic gates, enabling the collection and transmission of non-zero pairs to one side from the output of the first row OUT_1. If the IB_2/WB_2 and IB_4/WB_4 pairs are non-zero (i.e., both IB and WB have non-zero values), these pairs are assigned to OUT_1 and OUT_2, respectively, while OUT_3 and OUT_4 are filled with zero values.

The number of non-zero pairs is determined by summing the SEL signals from the MUX Input block. This count is then communicated to the controller, enabling the back-end unit to selectively process only the non-zero pairs. For instance, if 2 out of 4 pairs are non-zero, the back-end unit uses this information and requires only two cycles to process these pairs through the MAC unit, as shown in Fig. 4. In this manner, the system fully exploits both slice-level input and weight sparsity by omitting unnecessary computations on zero bit-slices in a fine-grained fashion.

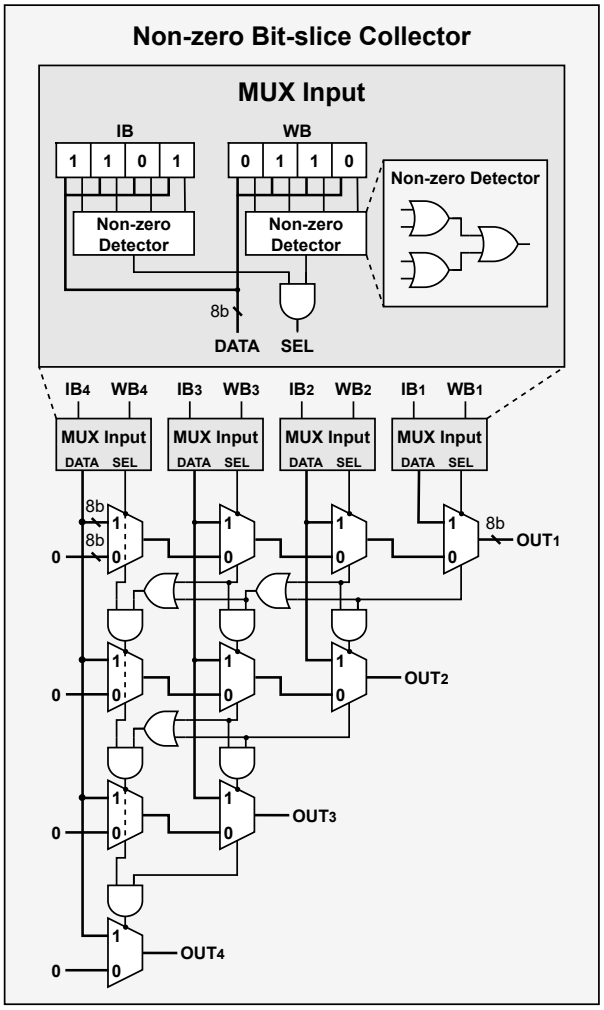


Fig. 6. Microarchitecture of the Non-zero Bit-slice Collector.

V. Evaluation

A. Methodology

Benchmarks. The proposed BADA architecture has been evaluated on both vision and NLP tasks. For the vision task, we employ VGG-16 [68], ResNet-50 [27], Inception-v3 [72], MobileNetV2 [63], and DeiT-Base/Small/Tiny [75], assessing their performance on the ImageNet dataset [40], a widely recognized benchmark for image classification. For NLP evaluation, we use RoBERTa [48], an optimized version of BERT [12], tested on the General Language Understanding Evaluation (GLUE) benchmark [78], which comprises eight diverse tasks: CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, and RTE. Additionally, the LLMs Llama-3.2 1B [14] and Gemma-2 2B [74] are evaluated using the HellaSwag benchmark [88], as detailed in Section V-G.

Software implementation. The model weights are initialized with pre-trained weights from PyTorch Image Models (TIMM) [81] for vision models and from Huggingface [82] for NLP models, using the PyTorch framework [57]. Quantization is performed using QAT with the LSQ quantizer [15]. Various data types, including 8-bit INT (INT8), 7-bit INT (INT7), and 8-bit RS-INT (RS-INT8), are used to quantize inputs and weights for all models, except for LLMs, which employ 16-bit data formats for inputs. The quantized vision models and RoBERTa are fine-tuned for 10 epochs on NVIDIA RTX 4090 GPUs with batch sizes of 256 and 64, respectively, whereas LLMs are fine-tuned for 5 epochs on NVIDIA A100 GPUs with a batch size of 32. A Stochastic Gradient Descent (SGD) optimizer with a cosine annealing learning rate scheduler [52] (excluding the warm-up scheme) is employed. The initial learning rate is set to 1e-3 for vision models, 4e-5 for RoBERTa, and 1e-6 for LLMs.

TABLE I
Specifications of Various Bit-slice Architectures

	Bit-fusion	Sibia	LUTein	**BADA**
Technology	28 nm	28 nm	28 nm	**28 nm**
Frequency (MHz)	250	250	250	**250**
Area (mm^2)	0.746	1.069	0.724	**1.268**
# of Multipliers	1536	1536	2048	**2048**
Precision of Multiplier	5b×5b	4b×4b	4b×4b	**4b×4b**
Data Type	INT8	INT7	INT7	**RS-INT8**
Peak Throughput (GOPS)	192.0	770.4	907.3	**2418.3**
Power (mW)	73.3	100.7	88.3	**109.3**
Energy efficiency (TOPS/W)	2.62	7.65	10.3	**22.1**
Area efficiency ($GOPS/mm^2$)	257.3	703.4	1252.7	**1907.2**

Hardware implementation. BADA is implemented at the Register-Transfer Level (RTL) using Verilog HDL and synthesized in a 28 nm CMOS process at a clock frequency of 250 MHz with Synopsys Design Compiler. The number of PEs in the BADA core, denoted by N, is set to 32, providing a total of 2048 multipliers, along with 64 KB of SRAM-based buffers and 3 KB of register files. Synopsys IC Compiler II is used for placement and routing, while Synopsys PrimeTime measures power consumption under operating conditions of 0.9 V and 125 °C. The BADA core occupies a total area of 1.268 mm^2 and consumes 109.3 mW.

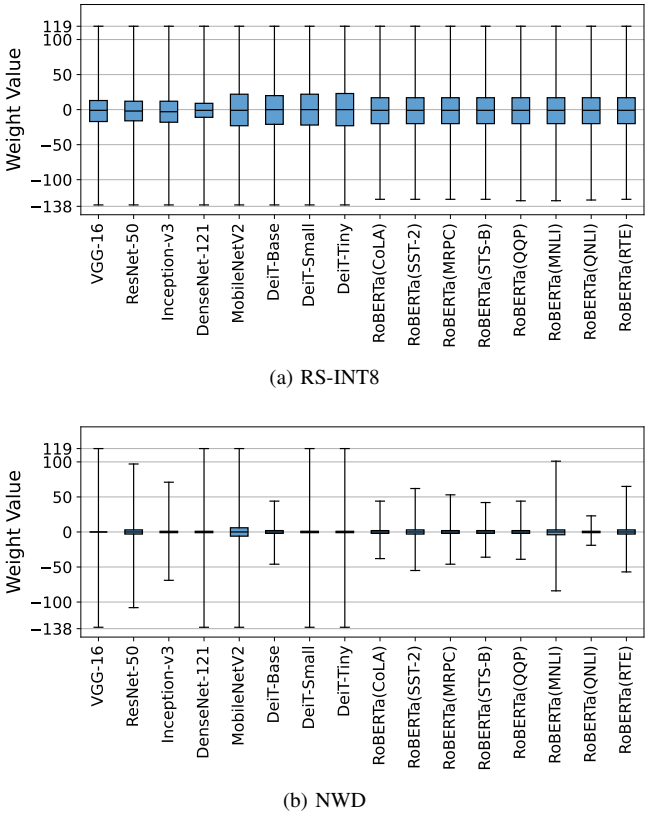
Accelerators for comparison. To evaluate the performance of BADA, we compare it against Bit-fusion [66], Sibia [33], and the state-of-the-art architecture LUTein [34]. Bit-fusion, Sibia, and LUTein employ INT8, INT7, and INT7 data types, respectively, for their bit-slice representations, while BADA utilizes RS-INT8. For a fair comparison, all accelerators are synthesized in 28 nm CMOS technology at a clock frequency of 250 MHz. A cycle-accurate simulator has been designed to provide both cycle counts and memory-access information, enabling performance comparisons between BADA and conventional accelerators. A summary of these accelerators is provided in Table I.

TABLE II
TOP-1 ACCURACY OF MODELS WITH VARIOUS DATA TYPES

Model	FP32	INT8	INT7	**RS-INT8**	**NWD (λ)**
VGG-16	72.97	72.94	72.70	**73.01**	**72.46 (1e-7)**
ResNet-50	79.63	78.33	77.72	**78.29**	**78.25 (1e-6)**
Inception-v3	76.99	76.89	76.86	**76.94**	**76.61 (1e-7)**
DenseNet-121	74.79	74.39	73.88	**74.02**	**73.98 (1e-7)**
MobileNetV2	72.20	72.03	71.88	**71.89**	**71.74 (1e-7)**
DeiT-Base	81.46	80.66	80.09	**81.01**	**80.29 (1e-7)**
DeiT-Small	79.31	79.11	78.19	**78.55**	**78.30 (1e-7)**
DeiT-Tiny	73.03	73.19	70.67	**73.14**	**72.12 (1e-7)**

TABLE III
RESULTS OF GLUE BENCHMARK WITH ROBERTA

Task	FP32	INT8	INT7	**RS-INT8**	**NWD (λ)**
CoLA (mc)	65.31	65.22	61.38	**64.62**	**64.52 (1e-4)**
SST-2 (acc)	93.69	93.69	92.78	**93.92**	**93.00 (1e-2)**
MRPC (acc)	90.44	90.93	88.73	**90.69**	**89.46 (1e-4)**
STS-B (pc)	90.70	88.89	74.58	**88.70**	**88.69 (1e-2)**
QQP (acc)	91.80	91.81	90.68	**91.69**	**90.96 (1e-6)**
MNLI (acc)	87.47	87.50	84.91	**87.31**	**86.49 (1e-8)**
QNLI (acc)	92.51	92.17	90.17	**92.17**	**91.65 (1e-6)**
RTE (acc)	79.78	80.51	68.59	**78.70**	**79.06 (1e-3)**



(a) RS-INT8

(b) NWD

Fig. 7. Box plots of weights for various models, including (a) RS-INT8 weights and (b) weights trained with scale regularization, forming an NWD. The boxes represent the second and third quartiles and the median, while the whiskers indicate the minimum and maximum values.

B. Accuracy Analysis

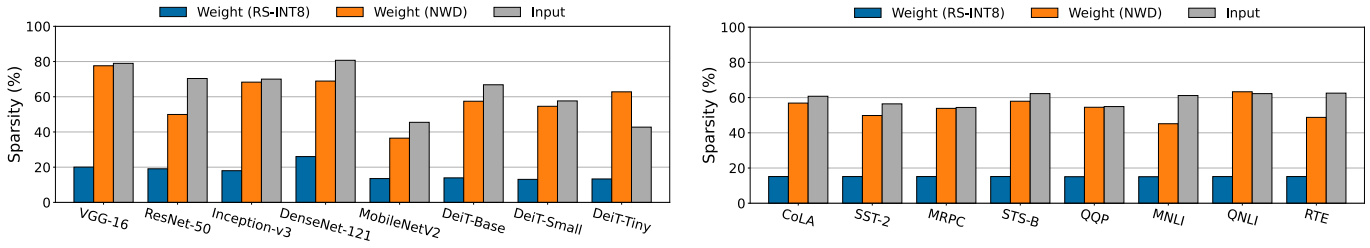
To ensure the effectiveness of the proposed algorithmic optimizations, it is essential to preserve the baseline accuracy of the quantized models. Two key factors that may influence model accuracy are adopting the novel integer representation RS-INT8 and training using the proposed scale regularization to form an NWD. Table II presents accuracy results for various vision models on the ImageNet dataset, evaluated using multiple data types, including 32-bit floating point (FP32), INT8, INT7, RS-INT8, and NWD, where NWD denotes RS-INT8 trained with scale regularization. Additionally, Table III shows results for RoBERTa evaluated on different GLUE tasks using these data types. The evaluation metrics for the GLUE tasks in Table III include *mc* (Matthews Correlation), *acc* (Accuracy), and *pc* (Pearson Correlation).

When the models are quantized from FP32 to INT8, a negligible average accuracy drop of 0.26% is observed. Meanwhile, RS-INT8 shows a modest average decrease of 0.49%, and in certain cases, it even surpasses the accuracy of INT8. In contrast, INT7, as utilized in Sibia and LUTein, results in a more substantial average accuracy drop of 6.16%, making it difficult to retain accuracy on NLP tasks. The λ values listed under the *NWD (λ)* column in Tables II and III are used when scale regularization is applied during model training with RS-INT8. As λ increases, the weight distribution narrows, albeit with a trade-off in model accuracy. Therefore, we choose the largest possible λ that preserves model accuracy comparable to RS-INT8, resulting in an NWD with an average accuracy drop of only 0.30%.

The results indicate that the range-shifted integer representation RS-INT8 maintains model accuracy comparable to that of the original integer representation INT8. Consequently, employing RS-INT8 in BADA achieves better accuracy than INT7 in Sibia and LUTein, despite both architectures incurring the same hardware overhead with 4-bit multipliers. Even when models are quantized using the QAT method, an aggressive quantization approach, INT7 exhibits a substantial accuracy drop of 11.23% on NLP tasks. Therefore, under less aggressive quantization methods such as PTQ, maintaining accuracy with INT7 becomes even more challenging, making RS-INT8 the more favorable option.

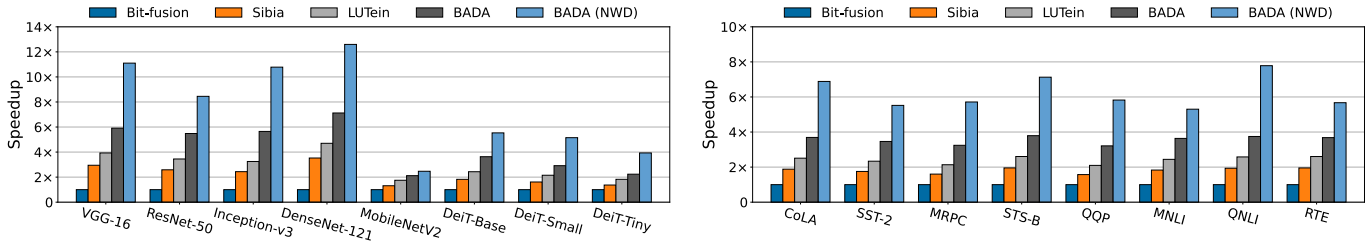
C. Sparsity Analysis

The proposed scale regularization can be employed during model training with negligible accuracy degradation, effectively forming an NWD, as illustrated in Fig. 7. The box plots show the weight distributions of all layers in various DNN models, where each box represents the interquartile range (IQR), encompassing the second and third quartiles. A smaller box size corresponds to a narrower weight distribution. Comparing these box plots indicates that the weights trained with scale regularization (Fig.7b) exhibit a noticeably narrower distribution than the baseline weights (Fig. 7a) across all models, thereby forming an NWD.



(a) Vision models (b) NLP tasks

Fig. 8. Comparison of slice-level weight sparsity for RS-INT8 and NWD, along with average slice-level input sparsity across (a) vision models and (b) NLP tasks.



(a) Vision models (b) NLP tasks

Fig. 9. Speedup comparison among bit-slice accelerators for (a) vision models and (b) NLP tasks.

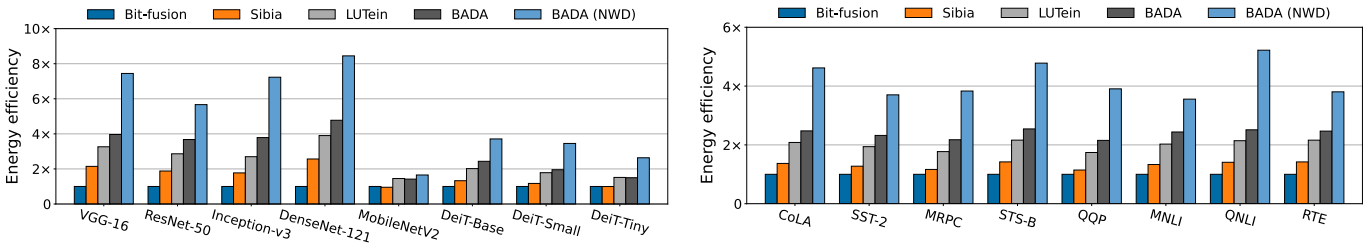
Additionally, an analysis of slice-level sparsity confirms that the resulting NWD increases slice-level weight sparsity. Slice-level sparsity refers to the sparsity derived from the proposed bit-slice representation, which is the output of the Bit-slice Unit in the PE. Fig. 8 shows the slice-level weight sparsity for baseline weights (RS-INT8) and weights trained with scale regularization (NWD), as well as the average slice-level input sparsity for various benchmarks. For slice-level input sparsity, the average results of both RS-INT8 and NWD are provided, as only negligible differences (an average of 0.4%) are observed between them. The results reveal a substantial increase in average slice-level weight sparsity across the benchmarks, rising from 16.13% to 56.64% when weights are formed as NWD. The most significant improvement appears in VGG-16, increasing from 20.0% to 77.6%. In contrast, the average slice-level input sparsity across all benchmarks is 61.72%. This higher input sparsity, compared to weight sparsity, stems from the prevalence of zero and near-zero values in the input data, largely resulting from activation functions such as ReLU and GeLU.

D. Performance Analysis

Fig. 9 compares the normalized inference performance of various bit-slice accelerators on different benchmarks, using Bit-fusion as the baseline. The results indicate that Sibia and LUTein achieve average speedups of 2.01× and 2.67×, respectively, whereas BADA demonstrates significantly higher performance with an average speedup of 3.97× over Bit-fusion. This enhanced performance in BADA arises from its comprehensive exploitation of both slice-level input and weight sparsity. In contrast, Bit-fusion does not leverage sparsity, and Sibia and LUTein utilize either slice-level input or weight sparsity, but not both. Specifically, Sibia and LUTein primarily rely on slice-level input sparsity, which exhibits higher sparsity levels than weight sparsity, as shown in Fig. 8. Intuitively, their speedup is proportional to the degree of slice-level input sparsity. However, BADA achieves superior speedup because its performance gain is driven by both slice-level input and weight sparsity.

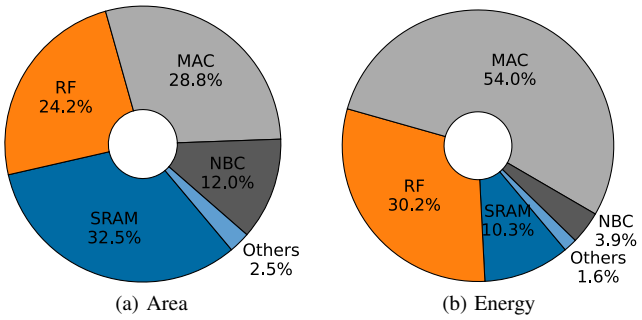
When BADA is evaluated using models trained with the proposed scale regularization to form an NWD, it demonstrates further performance improvements compared to its naive counterpart. The average speedup achieved by BADA rises from 3.97× (without NWD) to 6.86× (with NWD). The most significant gains are observed in CNN models such as VGG-16, ResNet-50, Inception-v3, and DenseNet-121, which achieve an average speedup of 10.73× due to high slice-level sparsity in both inputs and weights. In contrast, MobileNetV2 shows a lower speedup than other CNN models because of its lower sparsity. Additionally, Transformer-based models, such as DeiT and RoBERTa, exhibit diminished speedup, as these models involve more dense operations in fully connected layers and attention mechanisms, resulting in lower utilization (discussed in Section V-F).

Fig. 10 compares the energy efficiency of various accelerators, normalized to Bit-fusion. Sibia and LUTein achieve enhancements of 1.46× and 2.22× in energy efficiency, respectively. In contrast, BADA demonstrates the highest improvement ratio, with enhancements of 2.66× without NWD and 4.60× with NWD.



(a) Vision models (b) NLP tasks

Fig. 10. Energy efficiency comparison among bit-slice accelerators for (a) vision models and (b) NLP tasks.



(a) Area (b) Energy

Fig. 11. (a) Area breakdown and (b) energy breakdown of a PE Array.

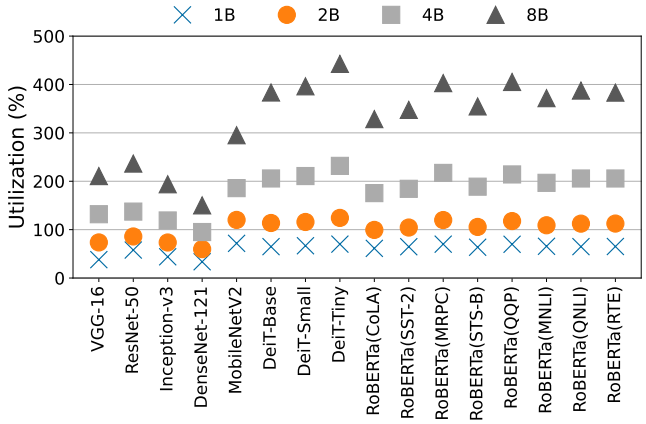


Fig. 12. Utilization of back-end units relative to the front-end unit with various output widths of NBCs across vision models and NLP tasks.

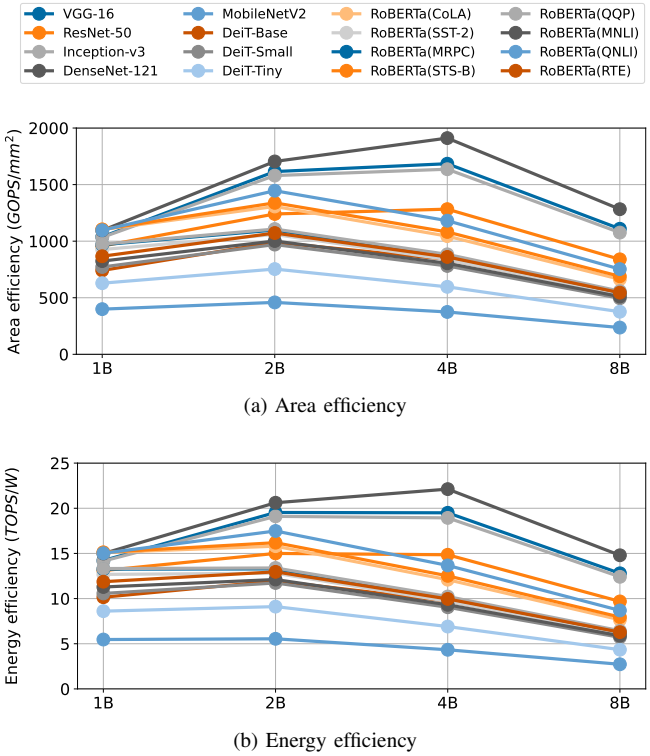
E. Area and Energy Analysis

Because the BADA core is designed to be versatile enough to support a variable number of PE Arrays, Fig. 11 shows the area and energy breakdown of a single PE Array. The area is dominated by memory components, including the Register File (RF) and SRAM, which account for 24.2% and 32.5%, respectively, totaling 56.7%, as shown in Fig. 11a. The remaining 43.3% is allocated to logic components, such as the NBC, the MAC unit, and other logic blocks. Notably, the NBC, characterized by numerous MUXs, occupies 12.0% of the total area, whereas the MAC unit, comprising a 4-bit multiplier and an accumulator, accounts for 28.8%. Other logic components, including the Bit-slice Unit, comprise 2.8%. Fig. 11b presents the breakdown of energy consumption, where the MAC unit dominates at 54.0% of the total energy. In comparison, the memory components of the RF and SRAM account for 30.2% and 10.3%, respectively, while the NBC and other logic elements consume 3.9% and 1.6%, respectively.

F. Design Space Exploration

BADA consists of a front-end unit that collects non-zero bit-slices and generates operands, as well as a back-end unit that consumes these operands. The output width of the NBC determines the throughput of operand generation in the front-end unit. Consequently, optimizing the output width of the NBC is crucial for balancing utilization between the front-end and back-end units, thereby achieving optimal overall performance. To identify the optimal configuration, we sweep the output width of the NBC from 1 byte to 8 bytes and measure the utilization of the back-end units relative to the front-end unit, as shown in Fig. 12. When utilization exceeds 100%, it indicates that operand generation in the front-end units outpaces consumption in the back-end units. The results show that an output width of 1 byte leads to under-utilization across all tasks, whereas 2 bytes maintain an average utilization of 103%. However, models with high slice-level sparsity in both inputs and weights, such as VGG-16, ResNet-50, Inception-V3, and DenseNet-121, require 4 bytes to achieve balanced utilization near 100%. These results indicate that utilization heavily depends on the slice-level sparsity of the models, requiring a higher NBC output width when models exhibit greater sparsity. Consequently, we choose 4 bytes as the output width of the NBC to optimize performance for models with significant sparsity. The second-best option is 2 bytes, which achieves balanced utilization in relatively dense models such as MobileNetV2 and Transformer-based models, including DeiT and RoBERTa.

Furthermore, to definitively determine the optimal output width of the NBC, we analyze area efficiency and energy efficiency across various output widths, as shown in Fig. 13. The results indicate that DenseNet-121 achieves the highest area and energy efficiency with an output width of 4 bytes, while output widths of 4 bytes and 2 bytes are optimal for sparse and dense models, respectively.



(a) Area efficiency

(b) Energy efficiency

Fig. 13. (a) Area efficiency and (b) energy efficiency of BADA with NBCs at various output widths across vision models and NLP tasks.

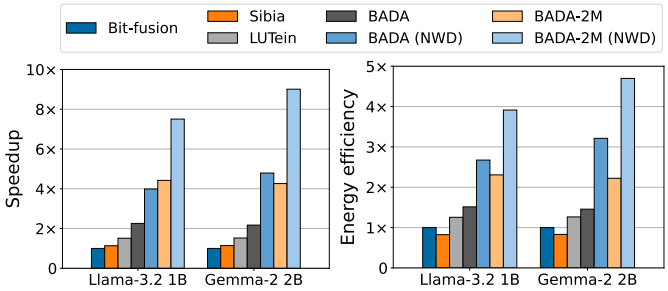
G. LLM Analysis

To further demonstrate the effectiveness of BADA with state-of-the-art LLMs, we evaluate its performance on Llama-3.2 1B and Gemma-2 2B, which are lightweight LLMs suited for edge and mobile devices. Table IV shows model accuracy on the HellaSwag benchmark using various combinations of data formats for weights and inputs, including FP32/FP32, INT8/INT16, INT7/INT16, RS-INT8/RS-INT16, and RS-INT8/RS-INT16 with NWD. When RS-INT8 is used instead of FP32, a negligible average accuracy drop of 0.09% is observed, whereas INT7 yields a slightly larger drop of 0.23%. Training with scale regularization at a λ of 1e-7 results in an average accuracy drop of 0.8% compared to FP32.

Fig. 14 presents the speedup and energy efficiency of the two LLMs with various accelerators, normalized to Bit-fusion. The results indicate that Sibia and LUTein exhibit modest average speedups of 1.13× and 1.52×, respectively. This limited performance gain arises from using SBR, which decomposes INT16 inputs into five bit-slices, whereas other accelerators split them into four bit-slices, resulting in additional computational overhead. Consequently, the energy efficiency of Sibia is 0.83× lower than that of Bit-fusion, whereas LUTein attains a 1.26× improvement in energy efficiency. In contrast, BADA achieves higher speedups of 2.22× without NWD and 4.39× with NWD, as well as energy efficiencies of 1.49× and 2.94×, respectively. These improvements arise from converting INT16 inputs into four bit-slices and exploiting slice-level sparsity in both inputs and weights.

TABLE IV
ACCURACY OF LLMS WITH VARIOUS DATA TYPES

Model	FP32	INT8	INT7	**RS-INT8**	**NWD (λ)**
Llama-3.2 1B	89.31	89.06	88.96	**89.18**	**88.38 (1e-7)**
Gemma-2 2B	94.09	94.07	93.98	**94.03**	**93.42 (1e-7)**



(a) Speedup

(b) Energy Efficiency

Fig. 14. (a) Speedup and (b) energy efficiency comparison among bit-slice accelerators for LLMs.

Although BADA exhibits notable performance, it shows an average utilization of 234.83%, both with and without NWD. Based on this observation, we conduct further analysis with a reconfigured version of BADA, termed *BADA-2M* in Fig. 14. In this variant, the back-end unit of the PE doubles the number of multipliers by incorporating an adder tree, leading to an average balanced utilization of 120.13% at the cost of a 1.26× larger area and 1.29× higher power consumption than the baseline BADA. Consequently, BADA-2M with NWD achieves significant gains in both speedup and energy efficiency, providing average improvements of 8.26× and 4.31×, respectively. Even without NWD, BADA-2M outperforms the baseline BADA, achieving an average increase of 1.96× in speedup and 1.52× in energy efficiency.

VI. DISCUSSION

A. Applicability and Versatility

Range-shifted integer. The range difference between the proposed RS-INT and the standard INT is minimal, particularly when the number of representation bits is small (e.g., $\Delta = 8$ for an 8-bit RS-INT). This slight variation results in negligible accuracy loss and, in some cases, even improves accuracy across various DNNs, including LLMs, as detailed in Sections V-B and V-G. Moreover, deploying RS-INT incurs overhead comparable to that of a standard INT, as both require quantization, and RS-INT introduces no additional complexity. Consequently, RS-INT is highly applicable and adaptable to a broad range of DNNs.

Scale regularization. As analyzed in Section V-D, the baseline BADA outperforms conventional accelerators, and its performance can be further enhanced by applying the proposed scale regularization during model training to form an NWD. However, since scale regularization modifies the weight distribution and requires full model fine-tuning with QAT, it can be challenging in scenarios where data is unavailable or

when handling extremely large models, such as GPT-4 [1] and Llama-3 405B [14]. Future work will focus on developing methods to modify the weight distribution without requiring full fine-tuning, utilizing techniques such as Parameter-Efficient Fine-Tuning (PEFT) [11], [31] or PTQ [18], [85], including cases where weights are quantized to lower bit widths, such as 4-bit.

B. Memory Optimization

To enhance the energy efficiency of BADA, we employ a tiling method as a memory optimization strategy, which maximizes data reuse and minimizes memory accesses. While this paper primarily focuses on computational optimization, the energy efficiency of BADA can be further improved by incorporating advanced memory optimization techniques, such as compression schemes [33], [59]. These methods are complementary to the proposed architecture and can lead to additional energy savings.

VII. Related Work

Bit-slice and bit-serial architectures. Given its bit-slice architecture, BADA is most closely related to DNN accelerators that employ either a bit-slice approach [23], [33], [34], [36], [45], [62], [66] or a bit-serial design [2], [38], [41], [44], [64], [67]. Both bit-slice and bit-serial architectures offer flexibility by enabling the reconfiguration of bit precision and employing low bit-precision multipliers to handle higher-precision data. Bit-fusion [66] serves as the foundational bit-slice architecture that leverages bit reconfigurability, while BitBlade [62] refines the architecture from Bit-fusion by optimizing the placement of shifting logic, thereby improving area and energy efficiency. In addition to bit reconfigurability, studies such as HNPU [23], Sibia [33], LUTein [34], and LEOPARD [44] explore input and output sparsity in bit-slice and bit-serial accelerators to improve effective throughput and energy efficiency. HNPU focuses on leveraging slice-level input sparsity, whereas Sibia and LUTein dynamically employ either input or weight sparsity through a hardware monitoring mechanism. Moreover, Sibia and LUTein extend their sparsity support to encompass output sparsity. In contrast, LEOPARD exploits output sparsity by training the threshold value using a custom learning algorithm, while other accelerators that incorporate output sparsity rely on heuristic methods to determine the threshold. However, conventional bit-serial and bit-slice architectures do not simultaneously take advantage of both input and weight sparsity. To address this limitation, we propose BADA, which facilitates the utilization of both input and weight sparsity, thereby enhancing performance and energy efficiency.

Transformer accelerators. While Transformer models achieve exceptional performance across various NLP and vision tasks, they demand significant computational resources. Consequently, numerous recent studies [17], [21], [22], [53], [60], [71], [73], [79], [86], [87] have introduced accelerators and architectures to enhance the inference speed of Transformer models, targeting the three main computation modules: Query, Key, Value (QKV) generation, attention, and the Feed-Forward Network (FFN). Most studies primarily focus on accelerating the attention module due to its quadratic complexity and predominant role in Transformer models with long inputs. Additionally, many of these studies leverage output sparsity, particularly within the attention module, which includes the softmax operation. However, FACT [58] found that other modules, including QKV generation and the FFN, incur higher computational overhead when the input length is relatively small. To address this, FACT [58] proposes an architecture that optimizes all three modules to improve the end-to-end performance of Transformer models. In contrast, BADA demonstrates superior performance on Transformer models by efficiently handling all three major computation modules, which primarily involve GEMM operations, through a bit-slice approach and fine-grained sparsity exploitation.

Other DNN accelerators. A number of studies [3], [5], [7]–[9], [39], [46], [47], [54], [61], [65], [77], [84] have introduced domain-specific accelerators to enhance the performance of various DNN applications. In addition to adopting efficient dataflows, several works exploit sparsity to further improve performance [4], [10], [19], [20], [24], [26], [42], [50], [55], [69], [70], [89], [90]. These approaches leverage weight sparsity through pruned weights [25], [28], [30], [56], [80] and input sparsity, as many models employ ReLU activation functions [27], [32], [63], [68], [72]. The proposed architecture, BADA, fully utilizes both input and weight sparsity, thereby enhancing performance and energy efficiency.

VIII. Conclusion

We propose BADA, a novel *Bit-Slice Architecture for DNN Acceleration*, which features a front-end unit that generates and collects non-zero bit-slices and a back-end unit that processes these bit-slices. BADA fully leverages both slice-level input and weight sparsity in both coarse-grained and fine-grained manners by omitting computations for bit-slice chunks or pairs containing zero values in the front-end unit. Furthermore, BADA enables the processing of 8-bit data with a novel bit-slice representation, whereas conventional bit-slice architectures process only 7-bit or 6-bit data under the same hardware overhead. To further improve the performance of BADA, we introduce a novel regularization method called scale regularization, applied during DNN model training. This method narrows the weight distribution, substantially increasing slice-level weight sparsity, which BADA exploits for enhanced efficiency. With these algorithmic optimizations, BADA outperforms the state-of-the-art bit-slice architecture LUTein, achieving 2.67× higher throughput, 1.52× higher area efficiency, and 2.15× higher energy efficiency.

Acknowledgements

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No.RS-2024-00405495, Plug&Play (P&P) Chiplet Integration research center). The EDA Tools used in this work were supported by IDEC, Daejeon, South Korea.

REFERENCES

[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 382–394.

[3] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-neuron-free deep neural network computing," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 1–13, 2016.

[4] B. Asgari, R. Hadidi, T. Krishna, H. Kim, and S. Yalamanchili, "Alrescha: A lightweight reconfigurable sparse-computation accelerator," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 249–260.

[5] E. Baek, D. Kwon, and J. Kim, "A multi-neural network acceleration architecture," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 940–953.

[6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[7] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.

[8] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 2, pp. 292–308, 2019.

[9] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun *et al.*, "Dadiannao: A machine-learning supercomputer," in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2014, pp. 609–622.

[10] C. Deng, Y. Sui, S. Liao, X. Qian, and B. Yuan, "Gospa: an energy-efficient high-performance globally optimized sparse convolutional neural network accelerator," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1110–1123.

[11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[13] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2020.

[14] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[15] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," in *International Conference on Learning Representations*, 2020.

[16] U. Evci, Y. Ioannou, C. Keskin, and Y. Dauphin, "Gradient flow in sparse neural networks and how lottery tickets win," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, no. 6, 2022, pp. 6577–6586.

[17] H. Fan, T. Chau, S. I. Venieris, R. Lee, A. Kouris, W. Luk, N. D. Lane, and M. S. Abdelfattah, "Adaptable butterfly accelerator for attention-based nns via hardware and algorithm co-design," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 599–615.

[18] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[19] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. Vijaykumar, "Sparten: A sparse tensor accelerator for convolutional neural networks," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 151–165.

[20] S. Gudaparthi, S. Singh, S. Narayanan, R. Balasubramonian, and V. Sathe, "Candles: Channel-aware novel dataflow-microarchitecture co-design for low energy sparse neural network acceleration," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 876–891.

[21] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee *et al.*, "Aˆ 3: Accelerating attention mechanisms in neural networks with approximation," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 328–341.

[22] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "Elsa: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 692–705.

[23] D. Han, D. Im, G. Park, Y. Kim, S. Song, J. Lee, and H.-J. Yoo, "Hnpu: An adaptive dnn training processor utilizing stochastic dynamic fixed-point and active bit-precision searching," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 9, pp. 2858–2869, 2021.

[24] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: Efficient inference engine on compressed deep neural network," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 243–254, 2016.

[25] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding," in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.

[26] E. Hanson, S. Li, H. Li, and Y. Chen, "Cascading structured pruning: enabling high data reuse for sparse dnn accelerators," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 522–535.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[28] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "Amc: Automl for model compression and acceleration on mobile devices," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 784–800.

[29] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 1389–1397.

[30] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 1389–1397.

[31] E. J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[32] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.

[33] D. Im, G. Park, Z. Li, J. Ryu, and H.-J. Yoo, "Sibia: Signed bit-slice architecture for dense dnn acceleration with slice-level sparsity exploitation," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 69–80.

[34] D. Im and H.-J. Yoo, "Lutein: Dense-sparse bit-slice architecture with radix-4 lut-based slice-tensor processing units," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 747–759.

[35] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.

[36] J.-W. Jang, S. Lee, D. Kim, H. Park, A. S. Ardestani, Y. Choi, C. Kim, Y. Kim, H. Yu, H. Abdel-Aziz *et al.*, "Sparsity-aware and re-configurable npu architecture for samsung flagship mobile soc," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 15–28.

[37] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles *et al.*, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.

[38] P. Judd, J. Albericio, T. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial deep neural network computing," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.

[39] W. Kong, Y. Hao, Q. Guo, Y. Zhao, X. Song, X. Li, M. Zou, Z. Du, R. Zhang, C. Liu *et al.*, "Cambricon-d: Full-network differential acceleration for diffusion models," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 903–914.

[40] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.

[41] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "Unpu: A 50.6 tops/w unified deep neural network accelerator with 1b-to-16b fully-variable weight bit-precision," in *2018 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 2018, pp. 218–220.

[42] S. Li, E. Hanson, X. Qian, H. H. Li, and Y. Chen, "Escalate: Boosting the efficiency of sparse cnn accelerator with kernel decomposition," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 992–1004.

[43] Y. Li, S. Xu, B. Zhang, X. Cao, P. Gao, and G. Guo, "Q-vit: Accurate and fully quantized low-bit vision transformer," *Advances in Neural Information Processing Systems*, vol. 35, pp. 34 451–34 463, 2022.

[44] Z. Li, S. Ghodrati, A. Yazdanbakhsh, H. Esmaeilzadeh, and M. Kang, "Accelerating attention through gradient-based learned runtime pruning," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 902–915.

[45] C.-H. Lin, C.-C. Cheng, Y.-M. Tsai, S.-J. Hung, Y.-T. Kuo, P. H. Wang, P.-K. Tsung, J.-Y. Hsu, W.-C. Lai, C.-H. Liu *et al.*, "7.1 a 3.4-to-13.3 tops/w 3.6 tops dual-core deep-learning accelerator for versatile ai applications in 7nm 5g smartphone soc," in *2020 ieee international solid-state circuits conference-(isscc)*. IEEE, 2020, pp. 134–136.

[46] L. Liu, Z. Qu, L. Deng, F. Tu, S. Li, X. Hu, Z. Gu, Y. Ding, and Y. Xie, "Duet: Boosting deep neural network efficiency on dual-module architecture," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 738–750.

[47] S. Liu, Z. Du, J. Tao, D. Han, T. Luo, Y. Xie, Y. Chen, and T. Chen, "Cambricon: An instruction set architecture for neural networks," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 393–405, 2016.

[48] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[49] Z. Liu, Y. Wang, K. Han, W. Zhang, S. Ma, and W. Gao, "Post-training quantization for vision transformer," *Advances in Neural Information Processing Systems*, vol. 34, pp. 28 092–28 103, 2021.

[50] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2ta: Exploiting structured sparsity for energy-efficient mobile cnn acceleration," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 573–586.

[51] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *International Conference on Learning Representations*, 2018.

[52] I. Loshchilov and F. Hutter, "Sgdr: Stochastic gradient descent with warm restarts," *arXiv preprint arXiv:1608.03983*, 2016.

[53] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 977–991.

[54] H. Mo, L. Liu, W. Hu, W. Zhu, Q. Li, A. Li, S. Yin, J. Chen, X. Jiang, and S. Wei, "Tfe: Energy-efficient transferred filter-based engine to compress and accelerate convolutional neural networks," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 751–765.

[55] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "Scnn: An accelerator for compressed-sparse convolutional neural networks," *ACM SIGARCH computer architecture news*, vol. 45, no. 2, pp. 27–40, 2017.

[56] J. Park, S. Li, W. Wen, P. T. P. Tang, H. Li, Y. Chen, and P. Dubey, "Faster cnns with direct sparse convolutions and guided pruning," in *International Conference on Learning Representations*, 2016.

[57] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[58] Y. Qin, Y. Wang, D. Deng, Z. Zhao, X. Yang, L. Liu, S. Wei, Y. Hu, and S. Yin, "Fact: Ffn-attention co-optimized transformer architecture with eager correlation prediction," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.

[59] Y. Qin, Y. Wang, Z. Zhao, X. Yang, Y. Zhou, S. Wei, Y. Hu, and S. Yin, "Mecla: Memory-compute-efficient llm accelerator with scaling sub-matrix partition," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1032–1047.

[60] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, "Dota: detect and omit weak attentions for scalable transformer acceleration," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 14–26.

[61] B. Reagen, P. Whatmough, R. Adolf, S. Rama, H. Lee, S. K. Lee, J. M. Hernández-Lobato, G.-Y. Wei, and D. Brooks, "Minerva: Enabling low-power, highly-accurate deep neural network accelerators," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 267–278, 2016.

[62] S. Ryu, H. Kim, W. Yi, and J.-J. Kim, "Bitblade: Area and energy-efficient precision-scalable neural network accelerator with bitwise summation," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.

[63] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.

[64] S. Sharify, A. D. Lascorz, M. Mahmoud, M. Nikolic, K. Siu, D. M. Stuart, Z. Poulos, and A. Moshovos, "Laconic deep learning inference acceleration," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 304–317.

[65] S. Sharify, A. D. Lascorz, K. Siu, P. Judd, and A. Moshovos, "Loom: Exploiting weight and activation precisions to accelerate convolutional neural networks," in *Proceedings of the 55th Annual Design Automation Conference*, 2018, pp. 1–6.

[66] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, J. K. Kim, V. Chandra, and H. Esmaeilzadeh, "Bit fusion: Bit-level dynamically composable architecture for accelerating deep neural network," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 764–775.

[67] D. Shin, I. Choi, and J.-S. Yang, "Vit-slice: End-to-end vision transformer accelerator with bit-slice algorithm," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[68] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[69] X. Song, T. Zhi, Z. Fan, Z. Zhang, X. Zeng, W. Li, X. Hu, Z. Du, Q. Guo, and Y. Chen, "Cambricon-g: A polyvalent energy-efficient accelerator for dynamic graph neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 1, pp. 116–128, 2021.

[70] N. Srivastava, H. Jin, J. Liu, D. Albonesi, and Z. Zhang, "Matraptor: A sparse-sparse matrix multiplication accelerator based on row-wise product," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 766–780.

[71] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient softmax for transformers," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 469–474.

[72] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2818–2826.

[73] T. Tambe, C. Hooper, L. Pentecost, T. Jia, E.-Y. Yang, M. Donato, V. Sanh, P. Whatmough, A. M. Rush, D. Brooks *et al.*, "Edgebert: Sentence-level energy optimizations for latency-aware multi-task nlp inference," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 830–844.

[74] G. Team, M. Riviere, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ramé *et al.*, "Gemma 2: Improving open language models at a practical size," *arXiv preprint arXiv:2408.00118*, 2024.

[75] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation

through attention," in *International conference on machine learning*. PMLR, 2021, pp. 10 347–10 357.
[76] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
[77] M. Vilim, A. Rucker, Y. Zhang, S. Liu, and K. Olukotun, "Gorgon: Accelerating machine learning from relational data," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 309–321.
[78] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "Glue: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.
[79] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 97–110.
[80] T. Wang, K. Wang, H. Cai, J. Lin, Z. Liu, H. Wang, Y. Lin, and S. Han, "Apq: Joint search for network architecture, pruning and quantization policy," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2078–2087.
[81] R. Wightman, "Pytorch image models," https://github.com/rwightman/pytorch-image-models, 2019.
[82] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Huggingface's transformers: State-of-the-art natural language processing," *arXiv preprint arXiv:1910.03771*, 2019.
[83] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," *arXiv preprint arXiv:2004.09602*, 2020.
[84] Y. N. Wu, P.-A. Tsai, S. Muralidharan, A. Parashar, V. Sze, and J. Emer, "Highlight: Efficient and flexible dnn acceleration with hierarchical structured sparsity," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1106–1120.
[85] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
[86] H. You, Z. Sun, H. Shi, Z. Yu, Y. Zhao, Y. Zhang, C. Li, B. Li, and Y. Lin, "Vitcod: Vision transformer acceleration via dedicated algorithm and accelerator co-design," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 273–286.
[87] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.
[88] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "Hellaswag: Can a machine really finish your sentence?" in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
[89] S. Zhang, Z. Du, L. Zhang, H. Lan, S. Liu, L. Li, Q. Guo, T. Chen, and Y. Chen, "Cambricon-x: An accelerator for sparse neural networks," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.
[90] X. Zhou, Z. Du, Q. Guo, S. Liu, C. Liu, C. Wang, X. Zhou, L. Li, T. Chen, and Y. Chen, "Cambricon-s: Addressing irregularity in sparse neural networks through a cooperative software/hardware approach," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 15–28.